

Reviewer Pack — Minimal Order of Formal Contexts and Communication Complexit...

Entry 849bbcc90cef · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 22:12:22 UTC

Minimal Order of Formal Contexts and Communication Complexity Rank Variance Ratio

Entry ID: 849bbcc90cef **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 22:12:22 UTC

1. Conjecture

Field A (mathematical branch): Lattice Theory (Formal Contexts) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every instance of the communication complexity problem, the minimal order of the associated formal context is linearly correlated with the variance ratio of its matrix representation, such that $\text{min_order}(\text{FC}) = \Theta(\text{variance_ratio}(n))$, where n is the size of the instance.

Rationale (proposer's reasoning):

Formal contexts provide a structured way to represent information and relationships between elements. By studying the minimal order of formal contexts associated with communication complexity instances, we may uncover new insights into the structure of these problems. A linear correlation suggests that the complexity of the problem can be characterized by properties of its formal context representation.

Taxonomy category: Formal_Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4c708011138b05cd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given communication complexity instance with size n , we consider the associated formal context's minimal order ($\text{min_order}(\text{FC})$) and its matrix representation's variance ratio as our metrics. A result is supported if $\text{min_order}(\text{FC}) / \text{variance_ratio}(n)$ falls within $[0.5, 2]$ for at least 80% of the 30 seeds, and a result is falsified if this ratio exceeds 2 or falls below 0.5 for more than 20% of the seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "formal contexts" AND "communication complexity" AND "matrix rank" - "minimal order of formal contexts" AND "communication complexity" AND "variance ratio" - "lattice theory" AND "communication complexity problem" AND "formal context order"

Top relevant hits considered: - [s2:80088936d1fbb171777e5a2fb73bb8f3c78e7725] Computer-aided verification : 2nd International Conference, CAV '90, New Brunswick, NJ, USA, June 18-21, 1990 : proceedi - [s2:10227042be21868b9335544c53b390ec45ddb79a] Automated Deduction - CADE-16: 16th International Conference on Automated Deduction, Trento, Italy, July 7-10, 1999, Pro

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = i + max(range(i, n), key=lambda k: abs(A[k][i]))
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i + 1, n):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        return A

    def matrix_rank(A):
        A = gaussian_elimination(A)
        rank = sum(1 for row in A if any(row))
        return rank

    def variance_ratio(matrix):
        n = len(matrix)
```

```

    mean = sum(sum(row) for row in matrix) / (n * n)
    var = sum((x - mean) ** 2 for row in matrix for x in row) / (n * n)
    return var / n if var != 0 else float('inf')

def formal_context(instance):
    n = len(instance)
    FC = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if instance[i][j]:
                FC[i][j] = 1
                FC[j][i] = 1
    return FC

def communication_complexity_instance(n):
    instance = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        instance[i][i] = 0
    return instance

n_max = 40
instances_tested = 30
metric_values = []

for _ in range(instances_tested):
    n = random.randint(5, n_max)
    instance = communication_complexity_instance(n)
    FC = formal_context(instance)
    rank = matrix_rank(FC)
    var_ratio = variance_ratio(instance)

    if var_ratio == float('inf'):
        continue

    metric_values.append(rank / var_ratio)

mean_value = sum(metric_values) / len(metric_values)
conjecture_holds = 0.5 <= mean_value <= 2
counterexample = "" if conjecture_holds else f"Mean value: {mean_value}"

return {
    "metric_name": "Minimal Order of Formal Contexts and Variance Ratio",
    "metric_value": mean_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

```

results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={sum((r['metric_value'] - mean_value) ** 2 / len(results)) ** 0.5}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_48f27f95.py", line 93, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_48f27f95.py", line 67, in run_trial
    rank = matrix_rank(FC)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_48f27f95.py", line 33, in matrix_rank
    A = gaussian_elimination(A)
        ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_48f27f95.py", line 27, in gaussian_elimination
    factor = A[j][i] / A[i][i]
              ~~~~~^~~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the division by zero error in the code to ensure that tests can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13587
2	propose	ollama_remote	glm4:latest	0	0	11762
3	preregistration	ollama_remote	glm4:latest	0	0	10109
4	novelty	ollama_remote	glm4:latest	0	0	11054
5	novelty	ollama_remote	glm4:latest	0	0	18418
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27914
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10891
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13489
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10370
10	judge	ollama_remote	glm4:latest	0	0	12308

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 139902 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/849bbcc90cef.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/849bbcc90cef.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/849bbcc90cef.tar.gz` (if generated)